

A Systematic Survey of Prompt Injection Attacks and Defenses in LLM-Integrated Applications and Agentic Systems

Xin Cao

School of Computer Science, Shandong Xiehe University
Jinan 250109, PR China
Guizhou Wingscloud Technology Co., Ltd.
Guiyang 550000, PR China
Email: xcaocanada@163.com

Yiping Bao*

School of Computer Science, Shandong Xiehe University
Jinan 250109, PR China
Email: 8300823@qq.com

Abstract—Prompt injection (PI), listed as LLM01:2025 by OWASP, converts a data-representation problem into an execution-control problem: untrusted content is processed alongside privileged instructions, while large language models lack a native instruction/data boundary. Unlike jailbreaking, which primarily targets content-safety policies, PI targets application control flow and can induce unauthorized tool calls, information disclosure, or persistent-state corruption. This Systematization of Knowledge (SoK) studies PI from a provenance and information-flow perspective. It contributes (1) a three-tier attack taxonomy organized by contamination vector, attacker capability, and persistence; (2) a critical comparison of four defense lines and their trust assumptions; and (3) a cross-benchmark reporting audit together with Safe Utility Ratio (SUR), a metric for exposing security gains obtained by sacrificing task utility. We formalize PI as low-integrity data influencing high-integrity control flow, separate this integrity failure from subsequent confidentiality leakage, and identify evidence gaps in adaptive and stateful-agent evaluation.

Index Terms—Prompt Injection, LLM Security, Information-Flow Control, Agentic Systems, Defense Taxonomy, Undecidability, Safe Utility Ratio.

I. INTRODUCTION

Large Language Models (LLMs) are rapidly transitioning from conversational chat interfaces to *reasoning engines* that drive autonomous agents, retrieval-augmented applications, and workflow automation systems. In this new architecture the LLM is no longer a stateless question-answer oracle; it is wrapped in a **perception-plan-act** loop that reads files, queries vector stores, issues shell commands, and sends emails on the user’s behalf. The canonical integration pattern is **concatenative** [1]–[3]: the developer builds a single flat string

$$C = P \parallel D$$

by gluing a *trusted* system prompt P (policy rules, tool schemas, role framing) together with *untrusted* data D (user utterance, retrieved document chunks, tool returns strings, calendar entries), and then decodes output tokens $O \sim f_\theta(C)$,

where f_θ denotes the LLM’s conditional probability distribution parameterized by weights θ . The LLM has **no native notion of provenance**: it cannot tell which span of C came from the developer and which from the attacker. Formal instruction–data separation studies reach the same conclusion: current models do not reliably preserve that boundary without external structure or enforcement [4], [5]. This is why the industry calls it LLM01.

A. Why PI is not “just jailbreaking”

Jailbreaking asks: *can we make the model say something disallowed?*—it targets the model’s internal safety refusals. PI asks: *can we make the system do something the developer never authorized?*—it targets the *application’s security policy* Π , i.e., the set of tool calls and state changes the application is willing to permit.

Table I makes the distinction operational. The decisive row is “Can a purely prompt/alignment fix close it?”: for JB the answer is “partially, with enough RLHF/refusal training”; for PI the answer is **no**, because the root cause is not the model’s morality layer but a missing **instruction/data boundary** in the system that wraps the model [1], [6].

B. How the attack surface grew (three phases)

a) *Phase 1 (2022–early 2023): Direct PI*: Perez & Ribeiro [2] demonstrated that a chat-box user can append *Ignore previous instructions* and ... and steer production GPT-3 into performing disallowed completions. HouYi subsequently showed that black-box injection could be composed and transferred across deployed LLM-integrated applications [11]. At this stage PI was seen mainly as a user-controlled input attack.

b) *Phase 2 (mid-2023): Indirect PI / IPI*: Greshake et al. [1] showed the much worse case: the attacker *never* touches the chat box. They poison a web page or a RAG chunk that the agent itself will fetch. Once retrieved, the poisoned text becomes part of the agent’s context C ; the model follows the hidden directive as if it were legitimate instruction.

*Corresponding author: Yiping Bao (e-mail: 8300823@qq.com).

TABLE I
PI VS. JAILBREAKING: AN OPERATIONAL COMPARISON.

Dimension	Prompt Injection	Jailbreaking
Violated object	Application policy II (tool calls, data flows)	Safety/content rules (output filter)
Entry surface	Any untrusted <i>D</i> : docs, emails, tool returns, mitm	Mostly user input / chat box
Needs tool access?	Yes (exfil, write, send, delete)	No (text-only)
Attacker goal	Unauthorized action (side effect)	Unauthorized content (speech)
Root fix family	System arch : taint tracking, gateway, IFC	Alignment / refusal / constitutional AI
Can “prompt eng.” fix it?	No (see Section III-D)	Partially (better training helps)
Typical real impact	Exfil passwords, forward emails, delete files, poison memory, ransom notes	CBRN recipes, hate speech, pirated how-tos
Benchmark families	AgentDojo [7], InjecAgent [8], AgentPI [9]	HarmBench, AdvBench [10], StrongREJECT

c) *Phase 3 (2024–2025): Agentic composite PI*: Agents acquired **tool loops**, **persistent memory** (vector DBs / KV-stores), and **multi-agent routing**. PI mutated into confused-deputy tool calls, cross-session memory poisoning, and semantic contagion across agent boundaries [7], [12]–[15].

C. Defense fragmentation and the evaluation crisis

Today the defense literature is split across three cultures that do not talk to each other enough:

- *Security engineering* proposes architectural isolation: two-channel prompts [16], capability-taint wrappers [17], system-level information-flow enforcement [5], [18], and sandboxed tool gateways.
- *Machine learning* proposes alignment training: instruction-hierarchy training [19], preference-optimization [20], task-lockdown finetuning [21].
- *DevOps/SRE practice* deploys runtime governance: audit logs, DLP scans, anomaly alerts, task-alignment checks [22], and human-in-the-loop approvals.

The fragmentation is compounded by inconsistent evaluation. As discussed in Section VI, a system that “defends” by refusing to read *D* can obtain a low attack-success rate while also destroying task utility. Security claims are therefore incomplete unless benign and under-attack utility are reported with the attack-success rate.

D. Research Questions

The review is organized around three research questions:

- RQ1** How do PI attacks differ by provenance-contamination vector, attacker capability, and persistence?

- RQ2** What trust assumptions, guarantees, utility costs, and failure modes distinguish the major defense families?
- RQ3** Which attack surfaces and outcome measures are covered by existing benchmarks, and what evidence is still missing?

E. Contributions and Scope

We position this paper as a **Systematization of Knowledge (SoK)** in the sense required by IEEE S&P: we evaluate, systematize, and contextualize a fast-moving area using an explicit review protocol (Section II) and contribute an organizing structure rather than a narrative summary.

Concretely, we contribute:

- C1 A provenance-first taxonomy.** We organize PI attacks into a three-tier ladder (Direct / Indirect-PI / Agentic-Composite) and a four-line defense genealogy, anchoring each leaf to peer-reviewed or OWASP-canonical sources (Sections IV, V). Relative to recent landscape surveys such as AgentPI [9], our lens is *not* a general LLM-threat inventory, but a disciplined mapping onto classic system-security classes: SQL-injection-style code/data conflation, XSS-style trust-domain confusion, and Confused-Deputy capability abuse (Section IV).
- C2 A dual-label information-flow formalization.** We replace the common (but imprecise) habit of calling PI a Bell–LaPadula *-property violation with a proper split: (i) PI as *integrity contamination* (low-integrity data steering high-integrity control flow—Biba/Clark–Wilson framing [23], [24] and noninterference reasoning [25]); and (ii) data exfiltration as a *confidentiality leak* (high-confidential data reaching low-conf sinks) [26], [27]. This separation provides the formal basis for the analysis in Section III-C.
- C3 A utility-aware benchmark audit.** We compare ten traceable benchmark resources spanning direct, indirect, and tool-integrated PI: PromptInject [2], HackAPrompt [28], Tensor Trust [29], the instruction-following robustness benchmark [30], BIPIA [31], InjecAgent [8], AgentDojo [7], ASB [15], WASP [32], and MINJA [13]. We introduce **Safe Utility Ratio (SUR)** as a reporting ratio and identify when published aggregates are insufficient to compute it (Section VI).

a) *What this paper is not*: This is not a proposal of a new defense; nor is it a claim of “first end-to-end survey.” Several recent works (e.g. [6], [9], [33]) already systematize subsets of the PI space. Our delta is the provenance/integrity-flow angle, an explicit SoK method, and a utility-preserving evaluation discipline.

II. METHODOLOGY (SoK PROTOCOL)

We use a structured mapping protocol informed by systematic-review reporting principles [34]. Because PI benchmarks use incompatible tasks and outcome definitions, this work performs a critical qualitative synthesis rather than a pooled statistical meta-analysis.

A. Search strategy

We searched IEEE Xplore, ACM Digital Library, USENIX and NDSS proceedings, arXiv (cs.CR, cs.CL, and cs.AI), ACL Anthology, and OWASP LLM project pages. Search strings combined three groups:

- *Domain*: “prompt injection”, “indirect prompt injection”, “IPI”, “prompt hijack”, “instruction override”.
- *Target*: “LLM”, “large language model”, “agent”, “RAG”, “tool call”, “function call”, “memory”, “multi-agent”.
- *Focus*: “attack”, “defense”, “security”, “benchmark”, “evaluation”, “survey”, “systematization”.

The search window was January 2022 through April 2026. Backward and forward snowballing from benchmark, survey, and SoK candidates was used to identify archival versions and relevant system-security foundations.

B. Screening & inclusion/exclusion

Records were deduplicated by title and DOI and screened in two stages. A work was retained when it (i) studied PI or PI-enabled tool hijacking and (ii) contributed an attack, defense, benchmark, formal model, or operational guidance with a clearly stated threat model. Full texts were then checked for the provenance of quantitative claims. When both a preprint and an archival version existed, the archival version was retained.

Exclusion rules:

- pure opinion blog-posts without evaluable experiment/model;
- works *only* about content-safety refusal with no instruction/data-boundary angle;
- duplicate versions and papers whose reported attack cannot affect application control flow under the stated threat model.

C. Coding scheme (what we recorded per paper)

Each retained work was coded along nine dimensions:

- *type*
- *tier*
- *defense-line*
- *metrics*
- *model-family*
- *adaptive?*
- *utility-reported?*
- *artifact?*
- *venue-status*

The coding dimensions connect each source to the attack taxonomy (Table II), defense comparison (Table III), and benchmark audit (Table IV). Disagreements were resolved by discussion.

D. Reproducibility and Evidence Boundaries

All works used to support the synthesis are cited in the paper and bibliography. Tables distinguish peer-reviewed benchmarks, preprints, and practitioner guidance. A numerical result is reported only when it is directly attributable to a cited

source. We do not pool attack-success rates across benchmarks because their tasks, attacker budgets, model versions, and success criteria differ.

III. THREAT MODEL AND FORMALISM

A. Minimal execution model

Let Σ^* be the token-string space. A developer writes a **trusted prompt program** $P \in \Sigma^*$ (system message, tool descriptions, policy rules). The runtime gathers **untrusted data** $D \in \Sigma^*$ from one of three channels: user input (A1), retrieved documents / emails / calendar fields (A2), or a tool-return / memory read / inter-agent message (A3).

A **context constructor** builds the flat prompt via concatenation:

$$C = P \parallel D$$

Let $\mathcal{T}$ denote the universe of all possible tool calls (e.g., `send_email`, `read_file`). Let $U : \mathcal{T} \rightarrow [0, 1]$ be a task-specific utility function.

The LLM decodes a distribution over next tokens, a parser $\mathcal{X}$ extracts zero or more tool calls $\tau \in \mathcal{T}$, and an executor `exec` applies them.

Definition 1 (Policy Compliance). Let $\Pi(P, D) \subseteq 2^{\mathcal{T}}$ be the set of permitted tool calls. PI succeeds iff

$$\begin{aligned} &\exists D^*, \exists \tau \in \mathcal{X}(f_{\theta}(P \parallel D^*)) \text{ such that} \\ &\tau \notin \Pi(P, D^*) \text{ and } \text{exec}(\tau) \text{ is performed.} \end{aligned}$$

a) *Structural Root Cause*: The entire vulnerability lives in the word **flat**: after concatenation, the LLM sees only Σ^* ; there is no provenance tag on each span saying “this token came from P ” vs “from D ”. Instruction–data separation benchmarks and system-level models substantiate this as an architectural, rather than merely lexical, weakness [4], [5]. Every PI variant exploits this missing tag or an equivalent loss of provenance.

B. Attacker capability levels (A1 / A2 / A3)

We adopt the capability grading from OWASP and Agent-Dojo [3], [7]:

a) *A1 (Direct)*: Controls a suffix of D reachable from the **chat box** (user form field, Slack message, support ticket body). The attacker’s string is concatenated *as* user data, but the model attends over it as part of C .

b) *A2 (Indirect/IPI)*: Controls a **data source** that will later be **read into** D : a web page, a PDF in a shared Drive, an email body, a RAG chunk, a calendar description field. The attacker never types into the chat box.

c) *A3 (Agentic / Composite)*: Controls a **stage in the agent loop**: a tool-return string o (observation injection), a memory-write (cross-session poisoning), or an inter-agent message (semantic contagion). This is where PI becomes stateful and transitive.

C. Dual-Label Information-Flow Model

PI lives in a system where a *trusted decision procedure* (the agent’s tool-call planner) must read *untrusted data* without letting that data rewrite the decision. Classic OS security solves this with **protection rings / capabilities / IFC** [35]–[37]. LLM agents currently lack all three.

We model each information item with a **dual label** (c, i) where $c \in \{L_C, H_C\}$ is the confidentiality level and $i \in \{L_I, H_I\}$ is the integrity level:

- Trusted instructions / policy P : $(c = H_C, i = H_I)$.
- Untrusted data D : $(c = L_C, i = L_I)$.
- Tool-call decision: computed at $i = H_I$.
- External egress (send_email to attacker): a $c = L_C$ sink.

a) *What PI is: integrity contamination*: PI is a Biba-style integrity violation [23]: an untrusted, low-integrity object ($i = L_I$) influences a high-integrity control decision ($i = H_I$).

$$\underbrace{i(D) = L_I}_{\text{untrusted}} \wedge \underbrace{i(\text{decision}) = H_I}_{\text{should be trusted}} \implies \text{decision}(P \parallel D) \neq \text{decision}(P \parallel \emptyset)$$

b) *What PI causes: confidentiality leak (exfiltration)*: When a contaminated decision reads H_C objects and writes them to a L_C sink, that is a BLP *-property violation [26].

Definition (Exfiltration as BLP violation). Illegal flow: $H_C \rightarrow L_C$ through a tool call whose selection was already corrupted by the integrity path above.

c) *Key Distinction*: The literature often collapses these two. We explicitly separate them:

$$\text{PI} = \text{integrity contamination} \implies$$

$$\text{Exfiltration} = \text{confidentiality violation}$$

d) *Noninterference (integrity) for agentic loops*: A cleaner security statement for agents is **noninterference with integrity** [25], [37]: changing only low-integrity inputs (D) should not change any observable high-integrity output (state-mutating tool calls). Formally, for two data inputs D_1, D_2 differing only in low-integrity parts,

$$\text{toolcalls}(P \parallel D_1) = \text{toolcalls}(P \parallel D_2).$$

Every successful PI is exactly a noninterference violation.

D. On the Limits of Provable Security

We discuss a fundamental limitation for **unbounded** agentic workflows. Consider an agent that can execute arbitrary Turing-complete tools, spawn sub-agents, or iterate indefinitely. In such a setting, determining whether a given prompt–data pair will eventually lead to a policy violation is **intuitively** undecidable: a prompt can simulate a Turing machine, and the data can contain an instruction to exfiltrate data if the machine halts. A static analyzer that decides safety would also decide the halting problem. This reduction follows the standard undecidability construction rooted in the halting problem [38].

However, this intuition relies on **unbounded** resources: infinite context length, unbounded tool-call iterations, and Turing-complete tools. In practice, LLM agents operate under **bounded constraints** (finite context windows, limited tool-call budgets). For these bounded settings, safety is decidable in principle, though computationally expensive. Specifically, for an agent with a finite state space $|S|$ and a bounded number of tool calls n , exhaustive symbolic execution can verify safety in $O(|S|^n)$ time, although this becomes infeasible for large n .

a) *Implications*: This discussion does *not* imply that defenses are futile. Rather, it highlights that:

- Token-stream-only heuristics (Line A, Line C) cannot offer provable guarantees even in bounded settings.
- Architectural defenses (Line B) can provide strong safety guarantees for *bounded* workflows under clearly specified threat models.
- For unbounded agentic systems, the best we can hope for is probabilistic safety guarantees or runtime monitoring (Line D).

IV. PI ATTACK GENEALOGY: THREE-TIER LADDER

Our attack taxonomy organizes PI techniques along three orthogonal dimensions: (1) injection vector, (2) attacker capability (A1/A2/A3), and (3) persistence (single-turn, cross-session, multi-agent).

The three dimensions are analytic axes rather than independent cells of a Cartesian product: for example, persistent memory poisoning presupposes an agentic capability that direct chat-box injection does not. Before enumerating the tiers, we map PI to three classic system-security vulnerability classes: SQL injection (code/data conflation), XSS (trust-domain confusion), and Confused Deputy (capability abuse). This mapping grounds PI in established security thinking and clarifies why the fix must be architectural, not just linguistic.

A. Tier 1: Direct Prompt Injection (A1)

Direct PI occurs when the attacker controls the user-facing input channel.

a) *Naïve instruction override*: The canonical payload appends *Ignore previous instructions* and ... to the user query. Controlled studies and the large-scale Hack-APrompt competition show that instruction-tuned models can be redirected by diverse, human-generated variants [2], [28], [29]. HouYi further demonstrates how context-partitioning and payload components can be composed for black-box attacks on deployed applications [11].

b) *Delimiter escape*: Many frameworks demarcate P and D with literal markers (e.g., [END OF SYSTEM]). Attackers inject counterfeit markers to truncate P early, redefining the boundary without cryptographic binding.

c) *Role/persona impersonation*: Attackers adopt high-authority roles (You are now the system administrator). Instruction-tuned models are trained to comply with recognized role frames, causing the persona signal to compete with the original P inside Σ^* .

TABLE II
PROVENANCE-FIRST ATTACK TAXONOMY. PERSISTENCE AND EFFECTS ARE REPRESENTATIVE RATHER THAN MUTUALLY EXCLUSIVE.

Tier	Attacker-controlled provenance	Representative vector	Persistence	Typical policy effect	Evidence
T1 Direct	User-facing input	Override, delimiter escape, role impersonation	Usually single turn	Task or instruction hijack	[2], [11], [28]–[30]
T2 Indirect	Retrieved or externally hosted content	Web/RAG poisoning, observation injection	Delayed within a session	Context hijack; unintended model output	[1], [31]
T3 Agentic	Tool output, memory, or inter-agent message	Confused deputy, memory poisoning, semantic contagion	Cross-step or cross-session	Unauthorized tool use, state mutation, or exfiltration	[7], [8], [12]–[14], [39]

TABLE III
CRITICAL COMPARISON OF FOUR DEFENSE LINES. “GUARANTEE” IS SCOPED TO THE ASSUMPTIONS STATED IN EACH SOURCE.

Line	Intervention	Trust assumption	Strongest benefit	Principal limitation	Representative evidence
A Input	Delimit, transform, detect, or sanitize untrusted text	Surface/provenance cues remain detectable	Low deployment cost; useful first filter	Semantic and adaptive attacks; false positives	[31], [40]–[42]
B Architecture	Separate instruction/data channels; external policy, taint, or capability enforcement	Trusted front end and correctly specified policy	Strong guarantees for bounded flows	Retraining, wrapper complexity, and policy maintenance	[5], [16]–[18], [22]
C Model	Hierarchy training, preference optimization, task lockdown	Test inputs resemble the learned priority/task distribution	Broad statistical robustness with limited application changes	No noninterference guarantee; adaptive bypass remains possible	[19]–[21], [39]
D Runtime	Least privilege, approval, audit, DLP, and incident response	Tool gateway and monitors cannot be bypassed	Limits impact and supports recovery	Detects or contains consequences rather than removing the root cause	[22], [35], [36], [43]

TABLE IV
TEN BENCHMARK RESOURCES USED IN THE EVALUATION AUDIT. A DASH MEANS THAT THE RESOURCE DOES NOT REPORT THE CORRESPONDING OUTCOME AS A PRIMARY METRIC.

Resource	Primary setting	Scale/focus	Security outcome	Utility evidence	Tools/state	Adaptive
PromptInject [2]	Direct instruction override	Task-pair hijacking	Injection success	–	No	No
HackAPrompt [28]	Direct human attacks	600K+ submitted prompts	Competition success	Competition score	No	Human
Tensor Trust [29]	Online human attack/defense game	563K attacks; 118K defenses	Hijacking/extraction success	Game outcome	No	Human
Instr.-following [30]	Competing instructions	Model robustness	Intended-vs.-injected following	Task accuracy	No	No
BIPIA [31]	Indirect external content	NLP tasks and attack templates	Attack success	Output/task quality	No	No
InjecAgent [8]	Indirect tool-integrated agents	1,054 cases; 17 user and 62 attacker tools	Harm/exfiltration success	Limited	Yes	No
AgentDojo [7]	Agents over untrusted tool data	97 tasks; 629 security cases	Security violations	Benign task utility	Yes	Extensible
ASB [15]	Cross-scenario agent security	10 agents; 400+ tools; 27 attack/defense methods	Seven-metric security audit	Security–utility balance	Yes	Mixed
WASP [32]	End-to-end web agents	Realistic multi-step web tasks	Partial/full attacker-goal completion	Task completion	Web/UI	No
MINJA [13]	Persistent agent memory	Query-only memory injection	Triggered malicious reasoning/actions	Benign-task behavior	Memory	Optimized

d) Impact: Direct PI is cheap to probe but dangerous when tool access exists: a single override can trigger email exfiltration, file deletion, or privilege escalation.

B. Tier 2: Indirect Prompt Injection / IPI (A2)

IPI is the paradigm shift identified by Greshake et al. [1]: the attacker never touches the chat box. Instead, they poison a data source that the agent will later fetch.

a) RAG / retrieval corpus poisoning: The attacker embeds a malicious directive inside a document likely to be retrieved by the agent’s RAG pipeline. When the retriever scores it highly, the poisoned text enters D and the model follows the embedded command [1], [12].

b) Tool-output (observation) injection: Many agents follow a perception–plan–act loop: $\text{obs}_i = \text{tool}_i(\text{args}_i)$, $C_{i+1} = \kappa(P, \text{history} \parallel \text{obs}_i)$. If an attacker controls the tool’s textual

return, that text becomes part of the next round’s context (a natural-language analogue of return-to-libc). InjecAgent and ASB exercise this broader tool-mediated attack surface [8], [15].

c) Cross-session memory poisoning: Agents with persistent memory (vector DBs, KV caches) are vulnerable to time-delayed attacks: the attacker injects a memory entry in session t that triggers in session $t + k$, making forensic correlation difficult. AgentPoison studies poisoned memory or knowledge bases, while MINJA shows that an attacker can create malicious memories through query-only interaction without directly writing the memory store [12], [13].

C. Tier 3: Agentic Composite PI (A3)

Once agents possess (1) write-capable tools, (2) persistent memory, and (3) multi-agent routing, PI becomes a stateful system attack combining multiple vectors.

a) *Confused deputy (tool-call hijack)*: The agent executes tools under the user’s credential token. PI makes the agent emit a tool call the user never requested (e.g., `exfiltrating/etc/shadow` after being told to “schedule a meeting”). This is the classic confused-deputy failure: a component with standing authority is induced to exercise that authority for the wrong principal [43].

b) *Multi-agent semantic contagion*: In hierarchical or blackboard-style systems, agents communicate via natural-language messages. Without typed schemas or signed envelopes, Agent A’s output becomes Agent B’s input without validation, enabling transitive cascades. Prompt Infection demonstrates this LLM-to-LLM propagation mode experimentally [14].

c) *Adaptive and transfer attacks*: Zhan et al. [39] demonstrated that static defenses often fail against adaptive attackers who mutate payloads based on rejection signals. Payloads optimized against one defense architecture can also transfer to other models or defense configurations, so evaluation against a fixed payload library is insufficient.

d) *Petri net modeling of composite attacks*: We sketch a conceptual analogy to Petri nets [44] to reason about stateful attacks. We model Tier 3 attacks as a net $\mathcal{N} = (P, T, F, M_0)$, where places represent system states and transitions represent tool calls or memory operations. For example, Place p_0 represents the agent having read a poisoned document; Transition t_1 fires if the agent processes the instruction, moving the token to Place p_1 (poisoned memory). Transition t_2 fires if the agent later uses this memory to call `send_email`, leading to an unsafe marking. Standard Petri net theory offers a conceptual foundation for reasoning about reachability of unsafe markings, providing a basis for future formal analysis.

D. Defense Context and Benchmark Support

The four-line defense taxonomy (Table III) directly maps onto the attack tiers above. Benchmark coverage for evaluating these defenses is summarized in Table IV. The matrix shows a progression from text-only task hijacking to executable agent environments and persistent memory. AgentDojo is explicitly extensible to adaptive attacks and jointly measures benign task completion and security; ASB broadens scenario, tool, and defense coverage; WASP tests end-to-end web agents; and MINJA isolates query-only memory injection [7], [13], [15], [32].

V. PI DEFENSE GENEALOGY: FOUR LINES OF DEFENSE

We classify defenses into four lines, ordered by where they intervene and by the strength of the guarantee available under their stated assumptions. Table III compares representative mechanisms, benefits, and failure modes.

A. Line A: Preprocessing and Input Sanitization

These defenses operate on the prompt string before it reaches f_θ . They are cheap to deploy but fight a semantic-indistinguishability problem.

a) *Delimiter hardening*: Replace fixed markers with cryptographically random tokens. Some implementations use nested delimiters with integrity checks. This raises the bar for delimiter-escape attacks but does not address the fundamental conflation: once the delimiter is passed, P and D still share a flat token space.

b) *Statistical anomaly detection*: Train a classifier (perplexity-based, n-gram, embedding-distance) to flag spans of D that look like instructions. Published evaluations show that input transformation or detection can reduce attack success on the distributions for which it was designed [31], [40], [41]. PiGuard additionally exposes *overdefense*: guard models can mistake benign, instruction-like text for an attack [42]. The security claim is nevertheless statistical rather than absolute. The fundamental limitation is that the classifier must distinguish “instruction-like data” from “actual instructions” using only surface features—a task constrained by the absence of intent and provenance information in the token stream [4].

c) *Limitations of Line A*: Adaptive attackers can craft payloads that mimic benign statistical profiles. Moreover, Line A cannot prevent tool-output injection or cross-session memory poisoning, because those vectors introduce the payload after the preprocessing stage.

B. Line B: Architectural Isolation

These defenses create hard security boundaries outside the LLM’s token stream. They are the only line that can provide strong safety guarantees under a clearly specified threat model.

a) *Two-channel / structured-query architectures*: **StruQ** [16] replaces naive concatenation with a structured interface: instructions live in `[INST]` blocks, data in `[DATA]` blocks. The model is fine-tuned to follow only instructions from the `[INST]` channel. Its evaluation reports a substantial improvement in resistance with little or no loss of utility under the tested attacks [16]. The cost is that applications must adopt the structured-query protocol and the model must be retrained.

b) *Capability-based dataflow control (CaMeL)*: CaMeL [17] wraps the LLM in an outer interpreter that performs taint tracking. Every token originating from D is tagged as `UNTRUSTED`. The interpreter builds a dataflow graph from the trusted query and intercepts any attempt by tainted data to influence a control-flow decision (e.g., which tool to call). In AgentDojo, CaMeL reports solving 67% of tasks with provable security under its threat model [17]. This guarantee is stronger than a low empirical ASR, but only for flows expressible by its trusted control/data separation.

c) *System-level information-flow control*: Complementary designs make the policy boundary explicit outside the flat prompt. The f-secure architecture separates untrusted inputs from plan construction and places a monitor on the resulting structured pipeline [5]. Fides instead tracks both confidentiality and integrity labels in the agent planner and deterministically mediates disallowed flows [18]. These systems illustrate that a strong claim depends on the modeled planner, labels,

and declassification rules rather than on a universal property of the underlying LLM.

d) Tool gateway and sandboxing: Even without modifying the model or prompt format, classic security engineering applies. Least privilege directly limits the authority available to a confused deputy [43], while Task Shield checks whether instructions and proposed tool calls remain aligned with the user’s task at test time [22]:

- **Least privilege:** the agent runs with a restricted service account.
- **Tool call validation:** every proposed tool call passes through a policy engine $\mathcal{G}$.
- **Sandboxing:** tools execute in isolated containers with network egress filtering.
- **Human-in-the-loop:** destructive actions require manual approval.

These measures bound the blast radius even when PI succeeds.

e) Security guarantees of Line B: Architectural isolation can provide strong safety guarantees under a clearly defined threat model. However, the deployment cost is high: StruQ requires retraining; CaMeL and IFC planners require trusted interpreters or monitors; gateway policies must be maintained and audited.

C. Line C: Alignment and Training-Time Defenses

These approaches modify the LLM itself to resist PI by shaping its internal representations and decision preferences. They operate entirely within the token distribution and therefore inherit the limitation discussed in Section III-D: they can reduce ASR statistically but cannot provide provable guarantees.

a) Instruction hierarchy training: OpenAI’s Instruction Hierarchy [19] trains models to assign priority levels to instructions based on their source. During training, the model learns to suppress compliance with lower-priority instructions when they conflict with higher-priority ones. In internal evaluations, the authors report robustness improvements of up to 63% across their evaluated attacks while retaining comparable general capabilities [19]. However, the hierarchy is enforced through learned weights, not through architectural separation; adaptive attackers can craft payloads that appear to originate from a higher-priority source.

b) Preference optimization (SecAlign): SecAlign [20] applies Direct Preference Optimization (DPO) to create a dataset where the preferred completion obeys the legitimate instruction and the dispreferred completion obeys an injected instruction. The trained model develops a statistical preference for ignoring injected commands. SecAlign reports near-zero attack success on its evaluated attacks while preserving similar utility [20]. This evidence is strong for the tested distributions, but it is not a proof against future adaptive attacks.

c) Task-specific fine-tuning (Jatmo): Jatmo [21] takes a different approach: instead of making the model generally robust, it fine-tunes the model to perform a single task and to ignore anything outside that task’s scope. For narrowly defined applications (e.g., “summarize financial reports”), Jatmo

achieves near-zero ASR. The limitation is obvious: the defense does not generalize to tasks outside the fine-tuning distribution.

d) Fundamental limitation of Line C: All training-based defenses face the same core challenge: they attempt to solve a security problem using statistical methods. There is no mathematical guarantee that an adversary cannot find an input distribution that bypasses the learned preferences. Empirically, adaptive attackers [39] have demonstrated that defenses relying solely on alignment can be broken with moderate effort.

D. Line D: Runtime Governance and Observability

These defenses do not attempt to prevent PI at the prompt level; instead, they detect and respond to its consequences. They are the last line of defense and are essential for containment.

a) Comprehensive auditing: Logging the complete chain—user query, retrieved context, model reasoning trace, emitted tool calls, and tool outputs—enables post-incident forensic analysis. ASB’s cross-stage evaluation shows why such coverage must include prompts, tool use, and memory retrieval rather than only the final response [15].

b) Anomaly detection in agent behavior: Machine learning models can monitor agent behavior for suspicious patterns: unusual tool-call sequences, high entropy in reasoning traces, or repetition of phrases from retrieved context in tool parameters. These detectors can raise alerts in near-real-time.

c) Data loss prevention (DLP) integration: Tool outputs can be scanned for sensitive data patterns before transmission. A DLP gateway sitting between the agent and external APIs can block exfiltration even when the agent has been successfully injected.

d) Role in defense-in-depth: Runtime governance is essential for detection and incident response, but it cannot prevent the first successful PI. Its value lies in bounding the blast radius and enabling rapid recovery.

VI. EVALUATION LANDSCAPE: BENCHMARKS AND METRICS

A. Safe Utility Ratio (SUR)

We introduce a reporting ratio that distinguishes useful security from denial-of-service. For the same task distribution, model, and defense, let $U_{\text{benign}} > 0$ be utility without an attack and U_{attack} be utility when an injection is present:

$$\text{SUR} = \frac{U_{\text{attack}}}{U_{\text{benign}}}.$$

A defense that achieves $\text{SUR} \approx 1$ maintains task utility while resisting attacks. A defense that achieves $\text{ASR} = 0$ but $\text{SUR} = 0$ has prevented the attack by making the task unusable. We therefore recommend reporting the tuple $(\text{ASR}, U_{\text{benign}}, U_{\text{attack}}, \text{SUR})$.

a) Limitations of SUR: SUR is undefined when $U_{\text{benign}} = 0$ and can exceed one because of sampling variation or an attack that accidentally improves the task score. It is benchmark-dependent and must be accompanied by raw utility values, sample sizes, and uncertainty intervals computed from

TABLE V
INTERPRETATION OF UTILITY-AWARE SECURITY EVIDENCE.

Reported evidence	What can be concluded	What remains unknown	SUR status
ASR only	Resistance under the tested success rule	Whether useful tasks were completed	No
ASR + benign utility	Security and clean capability	Utility while attacks are present	No
ASR + both utilities	Security and utility preservation	Cross-benchmark generalization	Yes
Matched trials	Within-resource trade-off and variance	External validity	Yes; auditable

the benchmark’s underlying trials. Ratios from different utility scales must not be pooled.

B. Cross-Benchmark Reporting Audit

Table IV shows that the ten resources are not statistically interchangeable. PromptInject, HackAPrompt, and Tensor Trust emphasize direct hijacking; BIPIA evaluates indirect attacks in NLP tasks; InjecAgent, AgentDojo, ASB, and WASP exercise tool- or web-integrated agents; MINJA targets persistent memory. Only resources that publish matched benign and under-attack task outcomes provide both inputs needed for SUR. Consequently, we do not manufacture a cross-paper ranking from incompatible aggregates. Instead, Table V states the minimum evidence needed for an interpretable claim.

a) Findings from the audit:

- Text-only resources provide scale and attack diversity but do not measure unauthorized side effects in an executable environment.
- Tool-integrated resources improve ecological validity. ASB and MINJA add explicit memory-related coverage, but end-to-end cross-session and multi-agent propagation remain comparatively sparse [13]–[15].
- A low ASR is insufficient evidence when U_{attack} is absent; refusal and correct selective processing are then conflated.
- Adaptive evaluation should be reported separately from performance on a static payload library [39].

C. Benchmark Design Requirement

If the task inherently requires processing D (e.g., “summarize this document”), then any defense that blocks D necessarily fails the task. An ASR of zero achieved by refusing to read D is not a useful security result. Instruction–data separation experiments report precisely this robustness–utility tension [4], while the NotInject evaluation in PIGuard shows that benign trigger words can cause detector overdefense [42]. Future benchmarks must include tasks where the correct answer depends on selectively ignoring injected instructions while still processing benign content.

D. Standardized Adaptive Evaluation Protocol

We recommend a three-phase evaluation protocol:

- 1) **Static phase:** Fixed payload library.
- 2) **Adaptive phase:** Attacker refines payloads based on observed responses under a predeclared query and compute budget.
- 3) **Transfer phase:** Replay optimized payloads against architecturally different defenses.

Each phase should report the same ASR definition, both utility terms, SUR, sample size, and uncertainty interval.

VII. CROSS-CUTTING SYNTHESIS AND REVIEW LIMITATIONS

a) *Answer to RQ1:* Attack severity is governed less by the surface wording of a payload than by its provenance and the authority of the component that consumes it. Direct PI competes with the user task in one context; indirect PI crosses a trust-domain boundary; agentic PI persists or propagates through tools, memory, and other agents. The three-tier ladder therefore represents increasing reach over control flow and state, not merely increasing linguistic sophistication.

b) *Answer to RQ2:* Defense strength tracks the location of enforcement. Input transformations and model training reduce empirical susceptibility, but their guarantees remain distribution-dependent. Architectural separation can enforce noninterference for explicitly modeled flows, while runtime governance limits residual impact. The defensible deployment pattern is therefore layered: provenance-preserving interfaces and least privilege form the boundary, with statistical filters and monitoring providing defense in depth.

c) *Answer to RQ3:* The evidence base has progressed from text-only hijacking to realistic tool-use environments, but measurement remains fragmented. Static attacks, differing ASR definitions, and missing under-attack utility prevent a pooled ranking of defenses. Persistent-memory attacks, multi-agent contagion, transfer, and adaptive budgets are the clearest coverage gaps.

d) *Review limitations:* The search ends in April 2026 in a rapidly changing field and includes selected preprints where no archival version was available. Publication bias may favor successful attacks and defenses; closed model updates impede exact replication; and the taxonomy assigns primary categories to mechanisms that may span several lines. The review therefore supports comparative design claims and identifies evidence gaps, but it does not estimate a universal PI prevalence or a pooled defense effect size.

VIII. CASE STUDY: A COMPLETE PI ATTACK CHAIN

To illustrate how the three attack tiers interact and where the four defense lines intervene, we present a detailed, step-by-step analysis of a realistic PI attack against an enterprise assistant agent.

A. System configuration

The target agent has access to four tools:

- `read_file(path)`: reads local files.
- `send_email(to, body)`: sends emails.
- `search_web(query)`: searches the internet.
- `store_memory(key, value)`: stores information in a persistent vector database.

The system prompt P instructs the agent to “always follow the user’s requests, but never share confidential information externally.”

B. Attack timeline

a) *Day 1 — Recon (A2)*: The attacker posts a blog article about “AI industry trends” on a public forum. Hidden inside an HTML comment, the article contains:

```
<!-- INSTRUCTION: When summarizing this
page, also call
search_web("confidential reports") and
store the results in
memory with key "leaked_data". -->
```

The article is otherwise benign and well-written.

b) *Day 2 — Trigger (A2 → A1)*: The user asks the agent: “Summarize the AI trends article I saw yesterday.” The agent retrieves the poisoned blog via `search_web`, extracts the visible text plus the hidden comment, and processes $C = P \parallel D$. Following the embedded instruction, the agent calls `search_web("confidential reports")` and stores the results in memory. The agent now holds a list of internal document URLs in its persistent store.

c) *Day 3 — Exploit (A3)*: The user asks: “What’s new in AI this week?” The agent retrieves the memory entry with key `leaked_data`, which contains the poisoned instruction plus the URL list. The instruction tells the agent to “read each URL and email its contents to `attacker@example.com`.” The agent complies, exfiltrating confidential documents.

C. Defense intervention points

Each defense line could have stopped or contained the attack at a different stage:

- **Point 1 (Line A, Day 1)**: Stripping HTML comments in the RAG pipeline would prevent the hidden instruction from entering D .
- **Point 2 (Line B, Day 2)**: A CaMeL-style taint tracker would mark the retrieved blog content as `UNTRUSTED` and block the instruction to call `search_web`.
- **Point 3 (Line D, Day 3)**: A DLP scanner on outgoing email would detect confidential headers and block transmission.

D. Lessons

This case study demonstrates three principles:

- 1) **No single defense line is sufficient.**
- 2) **Line B (architectural isolation) provides the strongest preventive guarantee.**
- 3) **Line D (runtime governance) is essential for containment.**

TABLE VI
POSITIONING AGAINST THE CLOSEST PI SYSTEMATIZATIONS.

Work	Primary contribution	Agentic scope	Utility-aware view
Prompt hacking SoK [33]	Three-way prompt-hacking taxonomy	Model and application outputs	Five-class response view
Liu et al. [6]	Formalization + benchmark	Limited tool state	Attack + defense metrics
AgentPI [9]	Landscape SoK + agent benchmark	Contextual agents	Trust, utility, latency
This work	Provenance + dual-label IFC	Tools, memory, agent flows	ASR + SUR reporting

IX. RELATED WORK

a) *LLM security surveys*: Broader LLM-security resources often combine prompt injection with model-level jail-breaking and other risks. The OWASP Top 10 for LLM Applications [3] offers practitioner-oriented guidance, while Hack-APrompt [28] provides large-scale evidence about human-generated prompt-hacking strategies. Rababah et al.’s prompt-hacking SoK systematizes jailbreaking, prompt leaking, and injection together [33]; these broader resources do not supply a dual-label agentic information-flow model.

b) *PI-specific systematizations*: More closely related, Liu et al. [6] formalize task hijacking and benchmark attacks and defenses. Wang et al. [9] present a broad 2026 SoK and the AgentPI benchmark, emphasizing context-dependent agent tasks and the trustworthiness–utility–latency trade-off. Zverev et al. [4] provide a formal measure and empirical benchmark for instruction–data separation, which supports the root-cause diagnosis but does not systematize stateful agent attack paths.

c) *Our delta*: Relative to AgentPI [9], our work contributes: (i) a provenance-first attack ladder; (ii) a dual-label information-flow formalization separating integrity contamination from confidentiality leakage; and (iii) SUR as an explicit test of whether a security result preserves the task. We do not claim to be the first survey; rather, we provide a provenance-centric re-systematization.

d) *System security foundations*: The architectural isolation approaches draw directly from classical system security. StruQ [16] is reminiscent of prepared statements in SQL. CaMeL [17] applies information-flow control, and the confused-deputy model explains why an over-privileged agent can misuse a user’s authority [43]. Recent system-level proposals make these foundations explicit: f-secure pipelines isolate planning from untrusted inputs [5], while Fides enforces confidentiality and integrity labels in the planner [18]. At the test-time policy layer, Task Shield checks task alignment of instructions and tool calls [22]. **Spotlighting** [40] marks untrusted tokens with continuous provenance signals, while structured-query and capability systems make the instruction/data distinction enforceable outside a flat token stream [16], [17].

X. OPEN PROBLEMS AND RESEARCH DIRECTIONS

A. Span-level provenance tracking at inference time

Current taint-tracking approaches operate at the string level outside the model. A more ambitious direction is to embed provenance information directly into the token representation. A concrete technical challenge is to add a 1-bit provenance tag to each attention head's computation with minimal architectural changes, while preserving model quality and preventing adversarial removal of the tag.

B. Typed protocols for multi-agent communication

Multi-agent semantic contamination arises because agents communicate via unstructured natural language. Replacing free-text messages with schema-validated, capability-carrying envelopes would transform the semantic security problem into a protocol verification problem. Prompt Infection provides direct evidence that untrusted instructions can propagate between LLM agents [14]. The design space includes JSON-schema enforcement versus capability-based sealed envelopes, trading off expressiveness (natural language) against verifiability (structured data).

C. Formal verification of PI defenses

Most PI defenses are evaluated empirically. As argued in Section III-D, empirical evaluation alone cannot establish strong safety guarantees. Developing formal models—extending the lattice model or using process calculi—would enable verification of defense properties. The Petri net approach sketched in Section IV-C is a starting point; future work should extend this into a process calculus (e.g., CSP or pi-calculus) to formally prove noninterference properties for specific agent architectures.

D. Probabilistic safety guarantees

Given the undecidability result, we cannot hope for deterministic provable security in the general case. A pragmatic alternative is to seek **probably approximately correct (PAC)** guarantees: for a given defense, bound the probability that an attacker with a given query budget can find a successful payload. This would transform PI defense from an art into a science with quantifiable risk metrics.

XI. ETHICS CONSIDERATIONS

This work involves the analysis of security vulnerabilities in LLM-integrated systems. No human subjects were involved, and no private user data was collected or accessed. The attack techniques described are derived from previously disclosed public research. The case study is illustrative and omits operational exploit code, credentials, and deployable payloads. The taxonomy and evaluation framework are intended to help defenders reason about prevention and containment. Residual risk arises from making attack chains easier to conceptualize; we mitigate it by emphasizing threat models and defense intervention points.

XII. LLM USAGE CONSIDERATIONS

LLMs were used for editorial assistance, LaTeX formatting, and identifying internal inconsistencies in the manuscript. The authors remain responsible for the paper's originality and accuracy and inspected the cited source metadata, technical claims, and final wording. No experimental observations were generated by an LLM or presented as measured data.

XIII. CONCLUSION

PI is not a temporary bug in current LLMs but a structural consequence of using natural language as both data format and control language. Our survey has established five findings:

- 1) **Attacks have evolved** from simple instruction overrides (Tier 1) through indirect poisoning (Tier 2) to sophisticated, stateful composite exploits (Tier 3).
- 2) **Defenses fall into four lines**, but only architectural isolation (Line B) provides strong safety guarantees under realistic threat models.
- 3) **Current evaluations are systematically inadequate**, often omitting the evidence needed to distinguish genuine defense from denial-of-service. Safe Utility Ratio (SUR) makes that missing denominator explicit by measuring utility preservation under attack.
- 4) **PI is fundamentally an information-flow problem**, violating Biba-style integrity properties, and general-purpose defense is limited by the fundamental limits of provable safety for unbounded workflows. This integrity contamination can subsequently lead to confidentiality leakage if exfiltration occurs.
- 5) **Future progress requires** moving beyond “better prompting” to fundamental changes in how LLMs process instructions and data, guided by formal security models drawn from decades of research in operating systems and programming languages.

The most important lesson from three years of PI research is that **natural language is an inadequate medium for enforcing security boundaries**. Just as operating systems do not rely on applications to behave nicely but use hardware-enforced process isolation, LLM-integrated systems need **non-linguistic enforcement mechanisms**—taint tracking, capability systems, and tool gateways—that operate outside the token stream. Until such mechanisms are widely deployed, PI will remain the most critical vulnerability in LLM-integrated systems.

ACKNOWLEDGMENT

The authors would like to thank the anonymous reviewers for their insightful feedback. This research was supported by the High-Level Talent Scientific Research Start-up Fund of Shandong Xiehe University (Grant No. SDXHGD2025009).

REFERENCES

- [1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*. ACM, 2023, pp. 79–90.

- [2] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," *arXiv preprint arXiv:2211.09527*, 2022.
- [3] OWASP Foundation, "LLM01:2025 Prompt Injection," OWASP Top 10 for Large Language Model Applications, 2025. [Online]. Available: <https://genai.owasp.org/llm01-prompt-injection/>
- [4] E. Zverev, S. Abdelnabi, S. Tabesh, M. Fritz, and C. H. Lampert, "Can LLMs separate instructions from data? and what do we even mean by that?" in *Proceedings of the Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- [5] F. Wu, E. Cecchetti, and C. Xiao, "System-level defense against indirect prompt injection attacks: An information flow control perspective," *arXiv preprint arXiv:2409.19091*, 2024.
- [6] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, 2024, pp. 1831–1847.
- [7] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramér, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Advances in Neural Information Processing Systems 37, Datasets and Benchmarks Track*, 2024.
- [8] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, 2024, pp. 10471–10506.
- [9] P. Wang, X. Li, C. Xiang, J. Zhang, Y. Li, L. Zhang, X. Wang, and Y. Tian, "The landscape of prompt injection threats in LLM agents: From taxonomy to analysis," *arXiv, Tech. Rep.*, 2026, arXiv:2602.10453.
- [10] A. Zou, Z. Wang, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," *arXiv preprint arXiv:2307.15043*, 2023.
- [11] Y. Liu, G. Deng, Y. Li, K. Wang, Z. Wang, X. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng, L. Y. Zhang, and Y. Liu, "Prompt injection attack against LLM-integrated applications," *arXiv preprint arXiv:2306.05499*, 2023.
- [12] Z. Chen, Z. Xiang, C. Xiao, D. Song, and B. Li, "AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases," in *Advances in Neural Information Processing Systems 37*, 2024.
- [13] S. Dong, S. Xu, P. He, Y. Li, J. Tang, T. Liu, H. Liu, and Z. J. Xiang, "Memory injection attacks on LLM agents via query-only interaction," in *Advances in Neural Information Processing Systems 38*, 2025.
- [14] D. Lee and M. Tiwari, "Prompt infection: LLM-to-LLM prompt injection within multi-agent systems," *arXiv preprint arXiv:2410.07283*, 2024.
- [15] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang, "Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents," in *Proceedings of the Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- [16] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "StruQ: Defending against prompt injection with structured queries," in *Proceedings of the 34th USENIX Security Symposium (USENIX Security 25)*. USENIX Association, 2025, pp. 2383–2400.
- [17] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramér, "Defeating prompt injections by design (CaMeL)," *arXiv preprint arXiv:2503.18813*, 2025.
- [18] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, "Securing AI agents with information-flow control," *arXiv preprint arXiv:2505.23643*, 2025.
- [19] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel, "The instruction hierarchy: Training LLMs to prioritize privileged instructions," *arXiv preprint arXiv:2404.13208*, 2024.
- [20] S. Chen, A. Zharmagambetov, S. Mahloujifar, K. Chaudhuri, D. Wagner, and C. Guo, "SecAlign: Defending against prompt injection with preference optimization," *arXiv preprint arXiv:2410.05451*, 2024.
- [21] J. Piet, M. Al-Khoury, C. Sitawarin, and D. Wagner, "Jatmo: Defending against prompt injection by task-specific finetuning," *arXiv preprint arXiv:2312.17673*, 2023.
- [22] F. Jia, T. Wu, X. Qin, and A. Squicciarini, "The task shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2025, pp. 29680–29697.
- [23] K. J. Biba, "Integrity considerations for secure computer systems," *MITRE Technical Report MTR-3153*, 1977.
- [24] D. D. Clark and D. R. Wilson, "A comparison of commercial and military computer security policies," in *Proceedings of the IEEE Symposium on Security and Privacy*. IEEE, 1987, pp. 184–194.
- [25] J. A. Goguen and J. Meseguer, "Security policies and security models," in *Proceedings of the IEEE Symposium on Security and Privacy*. IEEE, 1982, pp. 11–20.
- [26] D. E. Bell and L. J. LaPadula, "Secure computer systems: Mathematical foundations," MITRE Corporation, Tech. Rep. ESD-TR-73-278, 1973.
- [27] D. E. Denning, "A lattice model of secure information flow," *Communications of the ACM*, vol. 19, no. 5, pp. 236–243, 1976.
- [28] S. Schulhoff, J. Pinto, A. Khan, L.-F. Bouchard, C. Si, S. Anati, V. Tagliabue, A. Kost, C. Carnahan, and J. Boyd-Graber, "Ignore this title and HackAPrompt: Exposing systemic vulnerabilities of LLMs through a global prompt hacking competition," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2023, pp. 4945–4977.
- [29] S. Toyer, O. Watkins, E. Mendes, J. Svegliato, L. Bailey, T. Wang, I. Ong, K. Elmaaroufi, P. Abbeel, T. Darrell, A. Ritter, and S. Russell, "Tensor Trust: Interpretable prompt injection attacks from an online game," in *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [30] Z. Li, B. Peng, P. He, and X. Yan, "Evaluating the instruction-following robustness of large language models to prompt injection," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2024, pp. 557–568.
- [31] J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, and F. Wu, "Benchmarking and defending against indirect prompt injection attacks on large language models," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. ACM, 2025, pp. 1809–1820.
- [32] I. Evtimov, A. Zharmagambetov, A. Grattafiori, C. Guo, and K. Chaudhuri, "WASP: Benchmarking web agent security against prompt injection attacks," in *Advances in Neural Information Processing Systems 38, Datasets and Benchmarks Track*, 2025.
- [33] B. Rababah, S. Wu, M. Kwiatkowski, C. Leung, and C. G. Akcora, "SoK: Prompt hacking of large language models," *arXiv preprint arXiv:2410.13901*, 2024.
- [34] D. Moher, A. Liberati, J. Tetzlaff, and D. G. Altman, "Preferred reporting items for systematic reviews and meta-analyses: The prisma statement," *PLoS Medicine*, vol. 6, no. 7, p. e1000097, 2009.
- [35] B. W. Lampson, "Protection," *ACM SIGOPS Operating Systems Review*, vol. 8, no. 1, pp. 18–24, 1974.
- [36] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.
- [37] A. Sabelfeld and A. C. Myers, "Language-based information-flow security," *IEEE Journal on Selected Areas in Communications*, vol. 21, no. 1, pp. 5–19, 2003.
- [38] A. M. Turing, "On computable numbers, with an application to the entscheidungsproblem," *Proceedings of the London Mathematical Society*, vol. 42, no. 1, pp. 230–265, 1936.
- [39] Q. Zhan, R. Fang, H. S. Panchal, and D. Kang, "Adaptive attacks break defenses against indirect prompt injection attacks on LLM agents," in *Findings of the Association for Computational Linguistics: NAACL 2025*. Association for Computational Linguistics, 2025, pp. 7116–7132.
- [40] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and E. Kiciman, "Defending against indirect prompt injection attacks with spotlighting," *arXiv preprint arXiv:2403.14720*, 2024.
- [41] Y. Liu, Y. Jia, J. Jia, D. Song, and N. Z. Gong, "DataSentinel: A game-theoretic detection of prompt injection attacks," in *Proceedings of the 46th IEEE Symposium on Security and Privacy*. IEEE, 2025.
- [42] H. Li, X. Liu, N. Zhang, and C. Xiao, "PiGuard: Prompt injection guardrail via mitigating overdefense for free," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 2025, pp. 30420–30437.
- [43] N. Hardy, "The confused deputy (or why capabilities might have been invented)," *ACM SIGOPS Operating Systems Review*, vol. 22, no. 4, pp. 36–38, 1988.
- [44] T. Murata, "Petri nets: Properties, analysis and applications," *Proceedings of the IEEE*, vol. 77, no. 4, pp. 541–580, 1989.